

Sistemas Operativos

Licenciatura en Ciencias de la Computación.

FACEFQyN - Universidad Nacional de Río Cuarto

Práctico 6: Entrada/salida, filesystem y usuarios.

1. Dar la expresión regular en el comando dado abajo para listar los dispositivos de bloques en /dev.
`ls -l /dev | grep "regex here"`
2. Crear un archivo de 64MB inicializado en cero. Ayuda: Usar el comando dd que lea desde el dispositivo /dev/zero.
3. Escribir un comando que genere una contraseña de 16 caracteres codificado en base64 desde el dispositivo /dev/urandom. Ayuda: Ver los comandos head y base64.
4. Compilar el módulo (device driver) linux RTC (real time clock) dado siguiendo las instrucciones en README. Realizar el ejercicio como se pide.
5. Describir un sistema de archivos con la siguiente estructura de directorios y archivos

```
\
|-- docs
    |
    +-- os-book.pdf
```

Asumiendo que el archivo `os-book.pdf` ocupa dos bloques de datos.

- a. Usando una representación tipo FAT.
 - b. Usando una representación basada en *inodes*.
6. Dado el siguiente programa que lee el directorio corriente, extenderlo para mostrar el tipo y tamaño de cada archivo. Ayuda: Usar la función `stat(path, struct stat *buf)` (incluir `<sys/stat.h>`). El tipo del archivo se encuentra codificado en el campo `st_mode` (que también codifica los permisos). Ver `man stat`.

```

#include <stdio.h>
#include <dirent.h>

int main()
{
    DIR *dir = opendir(".");
    if (!dir) {
        perror("Unable to open directory");
        return 1;
    }
    struct dirent *entry;
    // Read each directory entry
    while ((entry = readdir(dir)) != NULL) {
        printf("File: %-20s, Inode: %lu\n", entry->d_name,
            (unsigned long)entry->d_ino);
    }
    closedir(dir);
    return 0;
}

```

Usuarios y control de acceso

1. Listar los usuarios y grupos del sistema. En Mac-OS usar el comando `dsc1` (directory services command).
2. Crear un directorio dentro de su directorio de trabajo. Ver los permisos que tiene.
 - a. ¿Qué significa que tenga permisos de ejecución?
 - b. Crear un archivo de texto dentro del directorio con el comando `touch`.
Analizar con qué permisos para el dueño, grupo y otros se creó.
 - c. Quitarle los permisos de lectura. Hacerlo con la sintaxis UGO y en octal.
 - d. Verificar si puede editar (modificar) el archivo. Explicar qué sucede.
 - e. Darle permisos de lectura nuevamente sólo para el dueño.
 - f. Intentar editarlo nuevamente.
3. Dado el siguiente programa que lista el *usuario efectivo* del proceso y su nombre (hacer `man getpwuid`).

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <pwd.h>

int main() {
    uid_t ruid = getuid();           // get real user id
    uid_t euid = geteuid();          // get effective user id
    struct passwd *pw = getpwuid(ruid); // get user data

    printf("Process Real User ID: %d, name: %s\n", ruid,
        pw->pw_name);
}

```

```

pw = getpwuid(euid);
printf("Process Effective User ID: %d, name: %s\n", euid,
      pw->pw_name);
return 0;
}

```

- a. Extenderlo para mostrar el shell que usa el *real user id*.
4. Extender el programa anterior para que
- a. Abrir el archivo creado en el ejercicio anterior para lectura y escritura y verificar que pudo abrirlo.
 - b. Cambiar dueño al archivo al usuario *root*.
 - c. Repetir el punto a. ¿Por qué el programa falla en abrir el archivo?
 - d. Probar nuevamente ejecutando el programa con *sudo*. ¿Funciona? Explicar por qué.
 - e. Cambiar el dueño del programa a *root* (con *sudo chown root <program>*).
 - f. Ejecutar nuevamente y explicar porqué el programa no puede acceder al archivo.
 - g. Ver si funciona ejecutándolo con *sudo*.
 - h. Encender el bit *suid* al programa (con *sudo chmod*).
 - i. Ejecutar el programa nuevamente con su usuario (no con *sudo*). Explicar porqué ahora puede abrir el archivo.